Bash System Health Automation Script — [6/14/2026]

OBJECTIVE: Develop a shell script to automate Linux system health reporting, practicing scripting fundamentals and system administration automation

ENVIRONMENT: Ubuntu 26.04 VM, Bash shell

SECTION 1: SCRIPT CREATION
- File created at:
  /home/user/scripts/system_health.sh
- Commands used:
  mkdir ~/scripts
  cd ~/scripts
  touch system_health.sh
  nano system_health.sh
- What I learned:
  I learned how Bash scripts are created and executed in Linux using a text editor and the Bash interpreter. The shebang line (*#!/bin/bash)* tells Linux which interpreter to use to execute the script. The operating system uses the Bash shell to process all commands contained within the file. Without this line, the system may not know which interpreter to use. I also learned that the *set -e* option improves reliability by causing the script to stop if a command fails. I used variables to store values such as timestamps and file paths, which made the script easier to maintain and reuse.

```
#!/bin/bash

set -e

if [ "$EUID" -ne 0 ]; then
    echo "Please run as root or with sudo"
    exit 1
fi

TIMESTAMP=$(date +"%Y-%m-%d_%H-%M-%S")
REPORT_DIR="/home/user/scripts/reports"
REPORT_FILE="$REPORT_DIR/health_report_$TIMESTAMP.txt"

mkdir -p "$REPORT_DIR"
```

SECTION 2: PERMISSIONS AND EXECUTION
- Command to make executable:
  chmod +x ~/scripts/system_health.sh
  ls -la ~/scripts/

- What I learned:

  I learned how Linux executable permissions work. Using *chmod +x*, it allows me to run it directly as a program or script from the command line. I then verified the permissions by using the *ls -la* command.

```
user@Ubuntu:~/scripts$ chmod +x ~/scripts/system_health.sh
user@Ubuntu:~/scripts$ ls -la ~/scripts/
total 12
drwxrwxr-x  2 user user 4096 Jun 14 08:42 .
drwxr-x--- 19 user user 4096 Jun 14 08:21 ..
-rwxrwxr-x  1 user user 1303 Jun 14 08:42 system_health.sh
```

SECTION 3: ERROR HANDLING

- Result when run without sudo:

```
user@Ubuntu:~/scripts$ ~/scripts/system_health.sh
Please run as root or with sudo
```

- Result when run with sudo:

```
user@Ubuntu:~/scripts$ sudo ~/scripts/system_health.sh
[sudo: authenticate] Password:
==========================================
       SYSTEM HEALTH REPORT
==========================================

--- DATE & TIME ---
Report Generated: Sun Jun 14 08:49:02 AM UTC 2026

--- SYSTEM IDENTITY ---
Hostname: Ubuntu
IP Address: 10.0.2.15 fd17:625c:f037:2:a00:27ff:fe47:d601

--- UPTIME ---
 08:49:02 up  5:10,  1 user,  load average: 0.42, 0.14, 0.10

--- DISK USAGE ---
Filesystem      Size  Used Avail Use% Mounted on
/dev/sda2        25G  6.8G   17G  30% /

--- MEMORY ---
               total        used        free      shared  buff/cache   available
Mem:           3.3Gi       2.3Gi       256Mi       116Mi       966Mi       1.0Gi
Swap:             0B          0B          0B

--- CPU ---
Model name:                           13th Gen Intel(R) Core(TM) i9-13900K

--- ACTIVE USERS ---

--- TOP 10 PROCESSES BY CPU ---
USER         PID %CPU %MEM    VSZ   RSS TTY      STAT START   TIME COMMAND
user        3193  1.3 10.8 5145664 377608 ?      Ssl  03:39   4:07 /usr/bin/gnome-shell --mode=ubuntu
user       10541  0.8 14.5 12132640 507660 ?     Sl   06:46   1:04 /snap/firefox/8107/usr/lib/firefox/firefox
```

- What I learned:

  I learned how Bash scripts can perform privilege validation using the EUID variable. The script checks if the current user is running with root privileges and exits if permissions are not available. Running the script without sudo triggered the validation check, while running it with sudo allowed the script to complete successfully. I also learned that privilege checking is an important form of error handling because it makes sure required permissions are available before beginning operations.

SECTION 4: SCRIPT COMPONENTS

- Report storage design:

The script uses */home/user/scripts/reports* instead of *$HOME* to ensure reports are saved in the intended user directory. When scripts are executed with *sudo*, *$HOME* resolves to */root*. This can cause files to be saved in unexpected locations.

- Output handling:

The script uses a curly brace block *{ }* to group all report output into a single stream. The output is then passed to *tee*, which displays the report on the screen while also saving it to a file.

- Automation features:

The *TIMESTAMP* variable stores the current date and time, which allows each report to receive a unique filename. The *mkdir -p* command automatically creates the reports directory if it doesn’t already exist. The *-t* flag sorts files by modification time, which makes it easy to locate the newest report.

```
user@Ubuntu:~/scripts$ ls -lt /home/user/scripts/reports/
total 24
-rw-r--r-- 1 root root 8254 Jun 14 08:59 health_report_2026-06-14_08-59-33.txt
-rw-r--r-- 1 root root 8334 Jun 14 08:49 health_report_2026-06-14_08-49-02.txt
```

- What I learned:

I learned how Bash scripts can automatically organize and manage output files without needing manual intervention. Using an absolute path ensured reports were saved in the correct location. The combination of a timestamp variable and automatic directory creation allowed the script to generate uniquely named reports while also maintaining a history of previous runs. These features showed how automation can improve consistency, reduce manual effort, and make system administration tasks easier to manage over time.

FINAL SCRIPT:

```
#!/bin/bash

set -e

if [ "$EUID" -ne 0 ]; then
    echo "Please run as root or with sudo"
    exit 1
fi

TIMESTAMP=$(date +"%Y-%m-%d_%H-%M-%S")
REPORT_DIR="/home/user/scripts/reports"
REPORT_FILE="$REPORT_DIR/health_report_$TIMESTAMP.txt"

mkdir -p "$REPORT_DIR"

{
echo "=========================================="
echo "       SYSTEM HEALTH REPORT"
echo "=========================================="
echo ""

echo "--- DATE & TIME ---"
echo "Report Generated: $(date)"
echo ""

echo "--- SYSTEM IDENTITY ---"
echo "Hostname: $(hostname)"
echo "IP Address: $(hostname -I)"
echo ""

echo "--- UPTIME ---"
uptime
echo ""

echo "--- DISK USAGE ---"
df -h /
echo ""

echo "--- MEMORY ---"
free -h
echo ""

echo "--- CPU ---"
```

```
lscpu | grep "Model name"
echo ""

echo "--- ACTIVE USERS ---"
who
echo ""

echo "--- TOP 10 PROCESSES BY CPU ---"
ps aux --sort=-%cpu | head -11
echo ""

echo "--- SSH SERVICE STATUS ---"
systemctl is-active ssh
systemctl is-enabled ssh
echo ""

echo "--- OPEN PORTS ---"
ss -tulnp
echo ""

echo "--- RECENT AUTH LOG ENTRIES ---"
tail -10 /var/log/auth.log
echo ""

echo "=========================================="
echo "       END OF REPORT"
echo "$(date)"
echo "=========================================="
} | tee "$REPORT_FILE"

echo ""
echo "Report saved as: $REPORT_FILE"
```

SCRIPT OUTPUT:

```
==========================================
       SYSTEM HEALTH REPORT
==========================================

--- DATE & TIME ---
Report Generated: Sun Jun 14 08:59:33 AM UTC 2026

--- SYSTEM IDENTITY ---
Hostname: Ubuntu
IP Address: 10.0.2.15 fd17:625c:f037:2:a00:27ff:fe47:d601

--- UPTIME ---
 08:59:33 up  5:21,  1 user,  load average: 0.04, 0.15, 0.12

--- DISK USAGE ---
Filesystem      Size  Used Avail Use% Mounted on
/dev/sda2        25G  6.8G   17G  30% /

--- MEMORY ---
               total        used        free      shared  buff/cache   available
Mem:           3.3Gi       2.5Gi       191Mi       116Mi       856Mi       861Mi
Swap:             0B          0B          0B

--- CPU ---
Model name:                              13th Gen Intel(R) Core(TM) i9-13900K

--- ACTIVE USERS ---

--- TOP 10 PROCESSES BY CPU ---
USER         PID %CPU %MEM    VSZ   RSS TTY      STAT START   TIME COMMAND
root       13653  1.3  0.2  25572  7036 pts/0    S+   08:59   0:00 sudo /home/user/scripts/system_health.sh
user        3193  1.3 10.8 5145656 377584 ?      Ssl  03:39   4:10 /usr/bin/gnome-shell --mode=ubuntu
user       10541  0.8 14.1 12132044 494064 ?     Sl   06:46   1:07 /snap/firefox/8107/usr/lib/firefox/firefox
user       11229  0.5 22.0 3492480 766236 ?      Sl   06:46   0:46 /snap/firefox/8107/usr/lib/firefox/firefox -contentproc -isForBrowser -prefsHandle
0:27827 -prefMapHandle 1:279684 -jsInitHandle 2:156136 -parentBuildID 20260404010525 -sandboxReporter 3 -chrootClient 4 -ipcHandle 5 -initialChannelId
 {fd3bb451-7171-4d41-a207-dbb71012d4c0} -parentPid 10541 -crashReporter 6 -crashHelper 7 -greomni /snap/firefox/8107/usr/lib/firefox/omni.ja -appomni
/snap/firefox/8107/usr/lib/firefox/browser/omni.ja -appDir /snap/firefox/8107/usr/lib/firefox/browser 7 tab
user       12446  0.1 10.4 2901356 362172 ?      Sl   07:00   0:14 /usr/bin/ptyxis --gapplication-service
```

```
root        1435  0.0  0.8 2181308 30760 ?       Ssl  03:38   0:10 /usr/lib/snapd/snapd
user       10869  0.0  3.4 2634676 121512 ?      Sl   06:46   0:02 /snap/firefox/8107/usr/lib/firefox/firefox -contentproc -isForBrowser -prefsHandle
0:27577 -prefMapHandle 1:279684 -jsInitHandle 2:156136 -parentBuildID 20260404010525 -sandboxReporter 3 -chrootClient 4 -ipcHandle 5 -initialChannelId
 {3b3c9a30-4ee4-410f-aad8-f4add2394979} -parentPid 10541 -crashReporter 6 -crashHelper 7 -greomni /snap/firefox/8107/usr/lib/firefox/omni.ja -appomni
/snap/firefox/8107/usr/lib/firefox/browser/omni.ja -appDir /snap/firefox/8107/usr/lib/firefox/browser 5 tab
user       10924  0.0  0.4 1876604 15768 ?       Sl   06:46   0:02 /usr/bin/snap userd
user       12165  0.0  2.0 2576812 71868 ?       Sl   06:56   0:01 /snap/firefox/8107/usr/lib/firefox/firefox -contentproc -isForBrowser -prefsHandle
0:45860 -prefMapHandle 1:279684 -jsInitHandle 2:156136 -parentBuildID 20260404010525 -sandboxReporter 3 -chrootClient 4 -ipcHandle 5 -initialChannelId
 {6c3704d2-0f6a-463a-b462-d33254600ef1} -parentPid 10541 -crashReporter 6 -crashHelper 7 -greomni /snap/firefox/8107/usr/lib/firefox/omni.ja -appomni
/snap/firefox/8107/usr/lib/firefox/browser/omni.ja -appDir /snap/firefox/8107/usr/lib/firefox/browser 14 tab
user       12214  0.0  2.0 2576812 71936 ?       Sl   06:56   0:01 /snap/firefox/8107/usr/lib/firefox/firefox -contentproc -isForBrowser -prefsHandle
0:45860 -prefMapHandle 1:279684 -jsInitHandle 2:156136 -parentBuildID 20260404010525 -sandboxReporter 3 -chrootClient 4 -ipcHandle 5 -initialChannelId
 {b244aadb-07c9-40f1-9070-4d51e61d160f} -parentPid 10541 -crashReporter 6 -crashHelper 7 -greomni /snap/firefox/8107/usr/lib/firefox/omni.ja -appomni
/snap/firefox/8107/usr/lib/firefox/browser/omni.ja -appDir /snap/firefox/8107/usr/lib/firefox/browser 15 tab

--- SSH SERVICE STATUS ---
active
enabled

--- OPEN PORTS ---
Netid State  Recv-Q Send-Q                     Local Address:Port  Peer Address:PortProcess
udp   UNCONN 0      0                                0.0.0.0:60229      0.0.0.0:*    users:(("firefox",pid=10541,fd=145))
udp   UNCONN 0      0                                0.0.0.0:56477      0.0.0.0:*    users:(("python3",pid=10120,fd=9))
udp   UNCONN 0      0                                0.0.0.0:5353       0.0.0.0:*    users:(("avahi-daemon",pid=1403,fd=12))
udp   UNCONN 0      0                              10.0.2.15:3702       0.0.0.0:*    users:(("python3",pid=10120,fd=10))
udp   UNCONN 0      0                        239.255.255.250:3702       0.0.0.0:*    users:(("python3",pid=10120,fd=8))
udp   UNCONN 0      0                                0.0.0.0:55013      0.0.0.0:*    users:(("firefox",pid=10541,fd=129))
udp   UNCONN 0      0                             127.0.0.54:53         0.0.0.0:*    users:(("systemd-resolve",pid=669,fd=18))
udp   UNCONN 0      0                          127.0.0.53%lo:53         0.0.0.0:*    users:(("systemd-resolve",pid=669,fd=16))
udp   UNCONN 0      0                                0.0.0.0:47407      0.0.0.0:*    users:(("firefox",pid=10541,fd=68))
udp   UNCONN 0      0                              127.0.0.1:323        0.0.0.0:*    users:(("chronyd",pid=1495,fd=4))
udp   UNCONN 0      0                                   [::]:5353          [::]:*    users:(("avahi-daemon",pid=1403,fd=13))
udp   UNCONN 0      0      [fe80::a00:27ff:fe47:d601]%enp0s3:3702          [::]:*    users:(("python3",pid=10120,fd=13))
udp   UNCONN 0      0                      [ff02::c]%enp0s3:3702           [::]:*    users:(("python3",pid=10120,fd=11))
udp   UNCONN 0      0                                      *:36709            *:*    users:(("python3",pid=10120,fd=12))
udp   UNCONN 0      0                                  [::1]:323           [::]:*    users:(("chronyd",pid=1495,fd=5))
tcp   LISTEN 0      4096                           127.0.0.1:631        0.0.0.0:*    users:(("cupsd",pid=1731,fd=7))
tcp   LISTEN 0      4096                             0.0.0.0:22         0.0.0.0:*    users:(("sshd",pid=1753,fd=3),("systemd",pid=1,fd=487))
tcp   LISTEN 0      4096                       127.0.0.53%lo:53         0.0.0.0:*    users:(("systemd-resolve",pid=669,fd=17))
```

```
tcp   LISTEN 0      4096                       127.0.0.54:53        0.0.0.0:*    users:(("systemd-resolve",pid=669,fd=19))
tcp   LISTEN 0      4096                             [::]:22           [::]:*    users:(("sshd",pid=1753,fd=4),("systemd",pid=1,fd=488))
tcp   LISTEN 0      4096                            [::1]:631          [::]:*    users:(("cupsd",pid=1731,fd=6))

--- RECENT AUTH LOG ENTRIES ---
2026-06-14T08:17:01.586622+00:00 Ubuntu CRON[12968]: pam_unix(cron:session): session closed for user root
2026-06-14T08:19:03.766863+00:00 Ubuntu gdm-password]: gkr-pam: unlocked login keyring
2026-06-14T08:30:02.186137+00:00 Ubuntu CRON[13273]: pam_unix(cron:session): session opened for user root(uid=0) by root(uid=0)
2026-06-14T08:30:02.540999+00:00 Ubuntu CRON[13273]: pam_unix(cron:session): session closed for user root
2026-06-14T08:40:55.559902+00:00 Ubuntu gdm-password]: gkr-pam: unlocked login keyring
2026-06-14T08:49:02.705390+00:00 Ubuntu sudo: pam_unix(sudo:session): session opened for user root(uid=0) by user(uid=1000)
2026-06-14T08:49:02.839314+00:00 Ubuntu sudo: user : TTY=/dev/pts/0 ; PWD=/home/user/scripts ; USER=root ; COMMAND=/home/user/scripts/system_health.sh

2026-06-14T08:49:04.884949+00:00 Ubuntu sudo: pam_unix(sudo:session): session closed for user root
2026-06-14T08:59:33.602552+00:00 Ubuntu sudo: pam_unix(sudo:session): session opened for user root(uid=0) by user(uid=1000)
2026-06-14T08:59:33.662858+00:00 Ubuntu sudo: user : TTY=/dev/pts/0 ; PWD=/home/user/scripts ; USER=root ; COMMAND=/home/user/scripts/system_health.sh


==========================================
       END OF REPORT
Sun Jun 14 08:59:34 AM UTC 2026
==========================================
```

CHALLENGES:

No major issues occurred during this project. The main consideration was ensuring the script saved reports to */home/user/scripts/reports* instead of using $HOME. Additionally, the generated report files were owned by *root* because the script was executed with elevated privileges. This caused issues after the project when attempting to upload files directly from the reports directory. Copying the file to the Desktop resolved the issue.

SKILLS DEMONSTRATED:

- Bash scripting
- Linux automation
- Script permission management
- Error handling
- System health monitoring
- Timestamped report generation
- Technical documentation
- File system organization
- Privilege escalation handling
- Process monitoring (ps aux)
- Service status verification
- Log file analysis